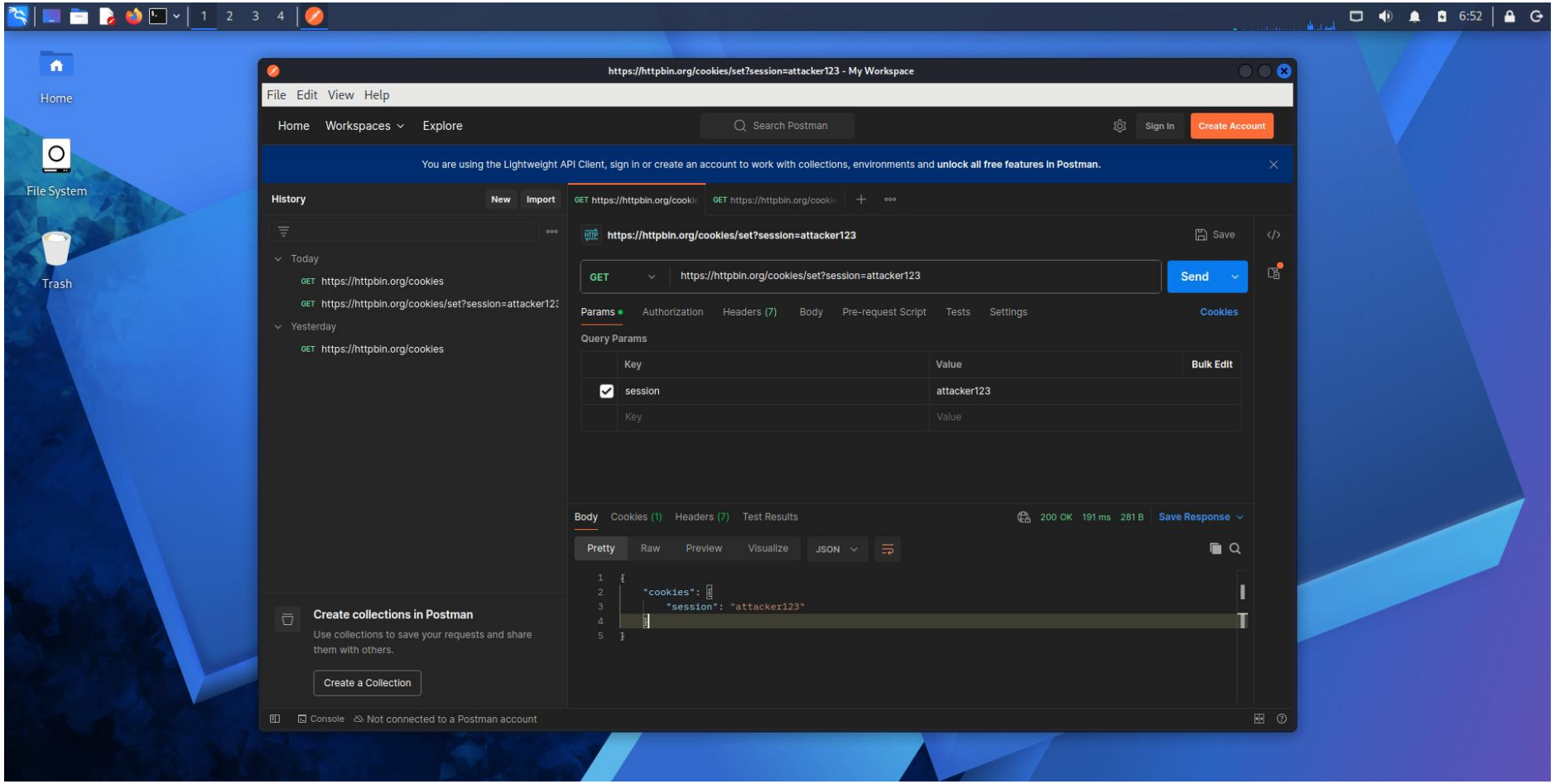
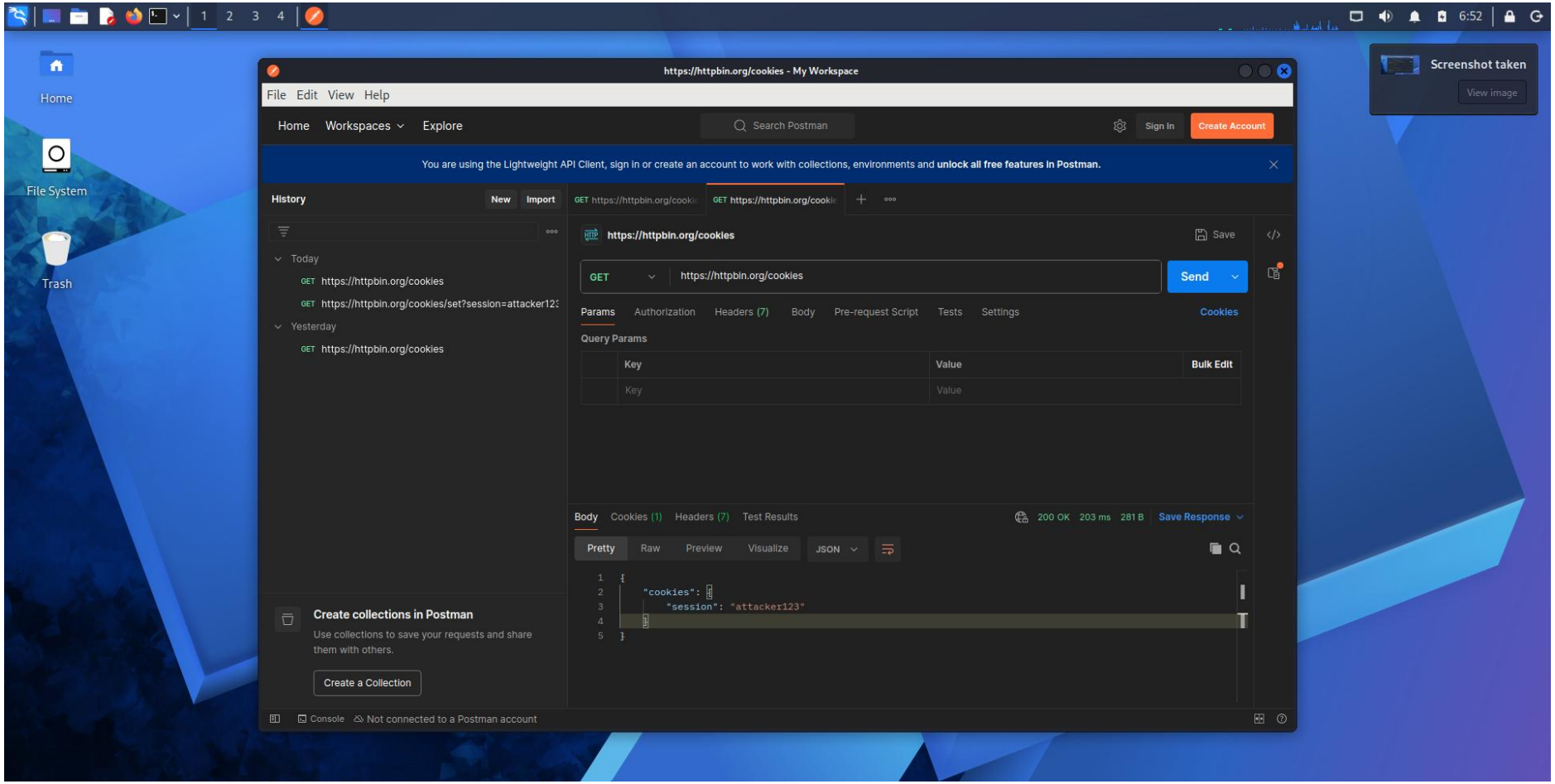

1 2 3 4
6:52
Home
File System
Trash
https://httpbin.org/cookies/set?session=attacker123 - My Workspace
File Edit View Help
Home Workspaces Explore
Search Postman
Sign In
Create Account
You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.
History
New
Import
Today
GET https://httpbin.org/cookies
GET https://httpbin.org/cookies/set?session=attacker123
Yesterday
GET https://httpbin.org/cookies
https://httpbin.org/cookies/set?session=attacker123
Save
GET
https://httpbin.org/cookies/set?session=attacker123
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Bulk Edit
session
attacker123
Key
Value
Body
Cookies (1)
Headers (7)
Test Results
200 OK
191 ms
281 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"cookies": {
"session": "attacker123"
}
}
Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection
Console
Not connected to a Postman account



Cookie Manipulation and Session Security

Results: I manually configured the cookie session=attacker123 in Postman and sent a request to the httpbin /cookies endpoint. The server returned the same value in its JSON response, confirming that the client-controlled cookie was transmitted and reflected by the server.

Security analysis: This experiment demonstrates cookie manipulation, but it does not establish an authentication bypass or session-management vulnerability because the endpoint does not validate authenticated sessions.

In a real application, insecure session management could allow attacks such as session fixation if a victim's authenticated session retains an identifier known to an attacker. Predictable or improperly validated session tokens could also expose accounts to unauthorized access.

Recommended mitigations :

1. Regenerate session identifiers after successful authentication and privilege changes.
2. Use cryptographically secure, unpredictable session identifiers.
3. Validate session identifiers and authentication state on the server.
4. Set appropriate Secure, HttpOnly, and SameSite cookie attributes.
5. Enforce session expiration and invalidate sessions on logout.

Conclusion: The test successfully demonstrated that a client can supply an arbitrary cookie value. Additional testing against an authorized application with a real login mechanism is required to determine whether its session-management controls are vulnerable.